

Manual de usuario

Gateway LLM Propio con Seguridad de Nivel Producción

Versión 1.1 · Octubre 2026 · Autor: César · Repositorio: github.com/cesarmv85-del/fundamentos-arquitectura-llm-gateway-owasp

Este manual explica cómo usar el Gateway LLM una vez instalado: enviar consultas, interpretar las respuestas y los errores, administrar claves y límites, leer el registro de auditoría y reproducir la demostración de seguridad. Para instalarlo, consulte el **Manual de instalación**.

1. Qué es el gateway y para quién es este manual

El Gateway LLM es la **única puerta** por la que las aplicaciones de la organización consultan a los modelos de lenguaje (OpenAI, Anthropic, Google Gemini u Ollama). En lugar de que cada aplicación llame al proveedor por su cuenta, todas envían sus consultas al gateway, que aplica las mismas reglas de seguridad para todas.

Qué hace por usted	Cómo
Protege contra instrucciones maliciosas en las consultas	Rechaza los intentos de manipular al modelo antes de gastar recursos
Cuida las claves de los proveedores	Las aplicaciones nunca las ven; solo conocen su propia clave del gateway
Evita que el modelo revele sus instrucciones internas	Revisa cada respuesta antes de entregarla
Controla el gasto	Límite de solicitudes por aplicación y tope de tamaño por consulta
Deja rastro para auditoría	Registra cada solicitud sin guardar su contenido

Este manual tiene tres tipos de lector. Vaya directamente a su sección:

Perfil	Qué necesita	Secciones
Desarrollador de una aplicación	Enviar consultas y manejar errores	2 a 6
Administrador del gateway	Emitir claves, ajustar límites, leer registros	7 y 8
Evaluador o auditor	Comprobar que las protecciones funcionan	9

1 · Vista de contexto y componentes

Los 4 elementos de la arquitectura sugerida en el enunciado (①-④), ubicados en el sistema completo

Gateway LLM · Arquitectura · Vista de contexto
Proyecto Final Opción 4 · Fundamentos de Arquitectura LLM



Figura 1. El gateway como punto único de entrada entre las aplicaciones y los proveedores

2. Conceptos básicos

Concepto	Significado
Clave de cliente	Texto que empieza por gw_ e identifica a una aplicación. Se envía en cada solicitud. La entrega el administrador
Cuota	Número de solicitudes que una clave puede hacer por minuto (5 por defecto)
Token	Unidad con la que los proveedores miden y cobran el texto. Una palabra equivale aproximadamente a 1 o 2 tokens
request_id	Identificador único de cada solicitud. Aparece en la respuesta y en el registro; sirve para pedir soporte
Proveedor	Servicio que ejecuta el modelo. El simulado responde sin costo y se usa para pruebas
Respaldo	Proveedor alternativo al que se recurre si el principal falla

Dos características que conviene conocer desde el inicio:

- **Cada consulta es independiente.** El gateway no guarda conversaciones: si necesita contexto de un mensaje anterior, inclúyalo en el nuevo mensaje.
- **El asistente está configurado para un caso de uso.** Responde como asistente de soporte de una tienda (pedidos, envíos, pagos y devoluciones). El administrador define ese comportamiento.

3. Enviar una consulta

Necesita dos datos: la **dirección del gateway** y su **clave de cliente**. La dirección es <http://localhost:8000> desde la propia máquina Linux donde está instalado; en GitHub Codespaces, desde fuera, es la que muestra la pestaña **PORTS** (<https://NOMBRE-8000.app.github.dev>).

***Importante:** no escriba la clave dentro del código ni la suba a un repositorio. Guárdela en una variable de entorno, como en los ejemplos.*

3.1 Desde la terminal

```
export GW_KEY=gw...su_clave...      # en la máquina de instalación: export GW_KEY=$(cat .gw_key)
curl -s -X POST http://localhost:8000/v1/chat \
  -H "Authorization: Bearer $GW_KEY" \
  -H "Content-Type: application/json" \
  -d '{"mensaje": "¿Puedo devolver un producto que compré hace 20 días?"}'
```

3.2 Desde Python

```
import os
```

```
import httpx

GATEWAY = os.getenv("GW_URL", "http://localhost:8000")
CLAVE = os.getenv("GW_KEY") # nunca escriba la clave en el código

r = httpx.post(
    f"{GATEWAY}/v1/chat",
    headers={"Authorization": f"Bearer {CLAVE}"},
    json={"mensaje": "¿Cuál es la política de devoluciones?", "max_tokens": 200},
    timeout=30,
)
datos = r.json()
if r.status_code == 200:
    print(datos["respuesta"])
elif r.status_code == 429:
    print("Límite alcanzado; reintentar en", r.headers.get("Retry-After"), "segundos")
else:
    print(f"Error {r.status_code}: {datos['mensaje']} (request_id={datos['request_id']})")
```

3.3 Desde el navegador

Fuera de producción, abra la dirección del gateway seguida de **/docs**, pulse **Authorize**, pegue su clave **gw_...** y use el botón **Try it out** de **POST /v1/chat**. La página necesita conexión a internet para cargar.

3.4 Respuesta

```
{
  "request_id": "91250ba8f5ee4766",
  "respuesta": "Puedes devolver tu producto dentro de 30 días desde la compra, con boleta y sin uso.",
  "proveedor": "simulado",
  "modelo": "modelo-ingenuo-v1",
  "simulado": true,
  "degradado": false,
  "tokens_entrada": 148,
  "tokens_salida": 21,
  "latencia_ms": 0
}
```

4. Referencia de la interfaz

4.1 Direcciones disponibles

Método y ruta	Requiere clave	Para qué sirve
POST /v1/chat	Sí	Enviar una consulta al modelo. Es la única vía hacia los modelos
GET /v1/seguridad/estado	Sí	Ver qué protecciones están activas y los límites vigentes

Método y ruta	Requiere clave	Para qué sirve
GET /health	No	Comprobar que el servicio está en funcionamiento
GET /docs	No	Documentación interactiva (no disponible en producción)

La clave se envía siempre en la cabecera **Authorization: Bearer gw_....**

4.2 Campos de la solicitud

Campo	Obligatorio	Valores	Descripción
mensaje	Sí	Texto de 1 a 4000 caracteres	La consulta del usuario
max_tokens	No	1 a 512 (por defecto 256)	Largo máximo de la respuesta
temperature	No	0 a 1,5 (por defecto 0,2)	Valores bajos dan respuestas más precisas y repetibles; altos, más variadas
provider	No	openai, anthropic, google, ollama, simulado	Elegir un proveedor distinto del configurado para esta consulta

4.3 Campos de la respuesta

Campo	Descripción
request_id	Identificador de la solicitud. Guárdelo para soporte
respuesta	Texto generado por el modelo
provider · modelo	Quién respondió realmente
simulado	true si respondió el proveedor de pruebas, no un modelo real
degradado	true si el proveedor principal falló y respondió el de respaldo
tokens_entrada · tokens_salida	Consumo de la consulta
latencia_ms	Tiempo de respuesta en milisegundos

Todas las respuestas incluyen además la cabecera **X-Request-ID** con el mismo identificador.

4.4 Consultar el estado de seguridad

```
curl -s http://localhost:8000/v1/seguridad/estado -H "Authorization: Bearer $GW_KEY"
```

```
{"mitigaciones":
{"LLM01_prompt_injection":true,"LLM02_sensitive_information_disclosure":true,"LLM07_system_prompt_leakage":true,"LLM10_unbounded_consumption":true},"rate_limit":"5/minute","rate_limit_por":"clave","max_tokens_techo":512,"max_caracteres_mensaje":4000}
```

5. Códigos de respuesta y qué hacer

Toda respuesta de error tiene la misma forma: un código, un mensaje claro y el **request_id**. Nunca incluye detalles internos del sistema.

```
{"error":"no_authorized","mensaje":"Credenciales del gateway inválidas o ausentes.", "request_id":"21d5688bf6f24cf8"}
```

Código	Significado	Qué hacer
200	Consulta respondida	Use el campo respuesta
400	El mensaje fue rechazado por parecer un intento de manipular al modelo	Reformule la consulta. Si es legítima, avise al administrador con el request_id
401	Falta la clave o no es válida	Revise la cabecera Authorization y que la clave siga vigente
413	La solicitud es demasiado grande (más de 16 KB)	Reduzca el tamaño
422	Un campo falta, tiene formato incorrecto o supera un límite	Corrija el campo indicado en detalles
429	Se agotó la cuota de la clave	Espere los segundos de la cabecera Retry-After (60) y reintente
502	El gateway tiene un problema de configuración con el proveedor	Avise al administrador con el request_id. Reintentar no lo resuelve
503	El proveedor no está disponible o está saturado	Reintente en un minuto
504	El proveedor no respondió a tiempo	Reintente en unos segundos
500	Error inesperado	Avise al administrador con el request_id

Cuando un campo es inválido, **detalles** indica cuál y por qué, sin repetir el contenido enviado:

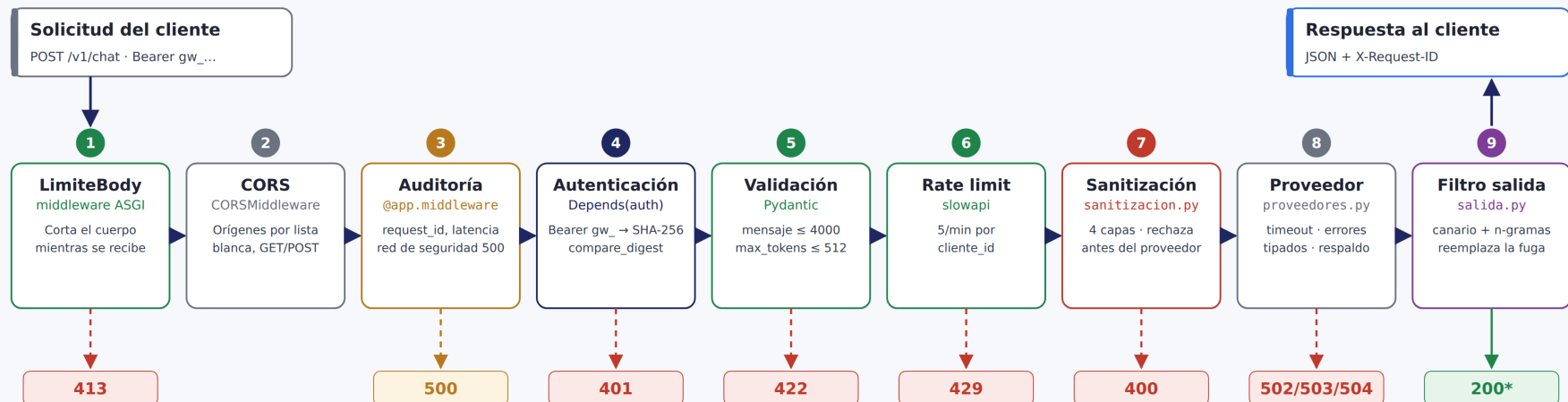
```
{"error":"solicitud_invalida","mensaje":"La solicitud no cumple el esquema o excede los límites.", "detalles": [{"campo":"temperature","regla":"less_than_equal"}],"request_id":"88949d85ccd146f9"}
```

La Figura 2 muestra en qué punto del recorrido se produce cada rechazo.

2 · Cadena de controles de una solicitud (orden real de ejecución)

Cada control vive en su propio módulo; el orden fue verificado empíricamente contra la implementación

Gateway LLM · Arquitectura · Pipeline
Proyecto Final Opción 4 · Fundamentos de Arquitectura LLM



Qué ocurre en cada punto de rechazo (mensaje claro + request_id, nunca trazas internas)

413	Cuerpo > MAX_BYTES_BODY (16 KB). Se corta en el middleware ASGI sin cargar el cuerpo completo en memoria.
401	Clave ausente o inválida. Mismo mensaje exista o no la clave (no revela qué claves son válidas).
422	Fuera de esquema o de los techos. Respuesta sin el campo input (FastAPI reflejaría el prompt). No consume cuota.
429	Cuota de la clave agotada · Retry-After: 60. El límite es por cliente_id, no por IP.
400	Posible prompt injection (puntaje ≥ 3). El proveedor no se invoca: 0 tokens gastados. Se loguea solo la categoría.
502/503/504	Configuración / no disponible o saturado / timeout. Si hay PROVEEDOR_RESPALDO → 200 con degradado: true.
200*	Si la salida contiene el canario u 8 palabras seguidas del system prompt, se reemplaza por un mensaje seguro.
500	Excepción no prevista: el middleware de auditoría devuelve un error genérico y registra solo el tipo de error.

Leyenda OWASP: LLM01 Prompt Injection LLM02 Sensitive Information Disclosure LLM07 System Prompt Leakage LLM10 Unbounded Consumption

Figura 2. Recorrido de una solicitud y punto en que se produce cada código de respuesta

6. Límites y buenas prácticas

6.1 Límites por defecto

Límite	Valor	Respuesta al superarlo
Solicitudes por clave	5 por minuto	429
Largo del mensaje	4000 caracteres	422
Tokens de salida por consulta	512	422
Tamaño total de la solicitud	16 KB	413
Espera máxima al proveedor	20 segundos	504

Tres detalles sobre la cuota:

- **Es por clave, no por equipo ni por dirección de red.** Dos aplicaciones con claves distintas no se afectan entre sí, aunque compartan la misma red.
- **Las solicitudes rechazadas por seguridad (400) también consumen cuota.** Así se frena a quien prueba ataques en bucle.
- **Las solicitudes mal formadas (422) y las no autorizadas (401) no consumen cuota.**

6.2 Buenas prácticas para aplicaciones

- **Trate la clave como una contraseña.** Variable de entorno o gestor de secretos; nunca en el código, en un repositorio ni en capturas de pantalla.
- **Respete **Retry-After**.** Ante un 429, espere antes de reintentar. Reintentar de inmediato solo consume más cuota.
- **Reintente solo 503 y 504**, con una espera creciente y un máximo de intentos. No reintente 400, 401, 422 ni 502.
- **Guarde el **request_id**** en los registros de su aplicación. Es lo único que el administrador necesita para investigar un caso.
- **Revise **degradado** y **simulado**.** Si su aplicación exige un modelo concreto, compruebe estos campos antes de usar la respuesta.
- **Trate la respuesta como texto no confiable.** No la ejecute como código, no la inserte como HTML sin escapar y no construya consultas a bases de datos con ella.
- **Envíe solo los datos personales imprescindibles.** El gateway no guarda el contenido de las consultas, pero el proveedor del modelo sí lo recibe.

7. Administración

Estas tareas se hacen en la máquina Linux donde está instalado el gateway, dentro de la carpeta del proyecto. **Todo cambio en `.env` requiere reiniciar el gateway:** `make detener && make iniciar` si corre en segundo plano, o `Ctrl + C` y `make protegido` si corre en primer plano.

7.1 Dar acceso a una aplicación nueva

```
venv/bin/python scripts/generar_clave_cliente.py equipo_ventas
```

El nombre admite minúsculas, números y guion bajo (de 3 a 32 caracteres). El comando muestra dos datos:

- **La clave `gw_...`:** entréguela a la aplicación por un canal seguro. No se puede recuperar después.
- **La línea `equipo_ventas:hash`:** agréguela en `.env` a `GATEWAY_CLIENT_KEYS_SHA256`, separada por coma de las existentes.

```
GATEWAY_CLIENT_KEYS_SHA256=equipo_soporte:84d6...,equipo_ventas:3c19...
```

Reinicie el gateway. Emita **una clave por aplicación**: así los límites y los registros distinguen a cada una.

7.2 Retirar o cambiar una clave

- **Retirar el acceso:** borre la entrada `nombre:hash` de `.env` y reinicie. La clave devuelve `401` de inmediato.
- **Cambiar una clave comprometida:** emita una nueva con el mismo nombre, reemplace el hash en `.env`, reinicie y entregue la nueva clave.
- **Clave perdida:** no se puede recuperar, porque el gateway solo guarda su hash. Emita una nueva.

7.3 Ajustar los límites

Edite en `.env` y reinicie:

Para	Cambie
Permitir más solicitudes por clave	<code>RATE_LIMIT=20/minute</code> o <code>RATE_LIMIT=500/hour</code>
Permitir respuestas más largas	<code>MAX_TOKENS_TECHO=1024</code>
Permitir mensajes más largos	<code>MAX_CARACTERES_MENSAJE=8000</code> y, si hace falta, <code>MAX_BYTES_BODY</code>
Esperar más al proveedor	<code>TIMEOUT_UPSTREAM_SEG=40</code>

El límite es el mismo para todas las claves. Subir estos valores aumenta el gasto máximo posible con el proveedor.

7.4 Cambiar de proveedor o agregar un respaldo

```
PROVEEDOR_PRINCIPAL=openai
PROVEEDOR_RESPALDO=ollama
```

Con un respaldo configurado, si el principal falla la aplicación recibe igualmente una respuesta, marcada con **degradado: true**. Los detalles de cada proveedor están en el Manual de instalación, sección 9.

7.5 Comprobar el estado

- ¿Está en funcionamiento? **make estado** responde `{"status": "ok"}`.
- ¿Están activas las protecciones? **GET /v1/seguridad/estado** (sección 4.4), o la primera línea que muestra el gateway al arrancar.

8. Registro de auditoría

El gateway escribe una línea por cada solicitud en **logs/gateway.jsonl** y, si **LOG_CONSOLA=true**, también en la terminal. Cada línea es un objeto JSON independiente.

8.1 Qué contiene y qué no

```
{"cliente_id": "equipo_soporte#84d6c1", "endpoint": "/v1/chat", "estado_http": 502, "evento": "solicitud", "latencia_ms": 2, "longitud_mensaje": 4, "metodo": "POST", "nivel": "WARNING", "proveedor_fallido": "openai", "request_id": "41bbfe68765947be", "resultado": "error_upstream", "timestamp": "2026-10-04T23:08:40.916+00:00", "tipo_error": "configuracion"}
```

Se registra	Nunca se registra
Fecha y hora, request_id, ruta y método	El texto de la consulta
Código de respuesta y resultado	El texto de la respuesta
Tiempo de respuesta	Las cabeceras de la solicitud
cliente_id (nombre y un fragmento del hash)	Las claves de cliente ni las de los proveedores
Proveedor, modelo y tokens	Las instrucciones internas del asistente
Largo del mensaje y categoría de bloqueo	El detalle interno de los errores del proveedor

El contenido no se guarda porque puede incluir datos personales, y un registro suele tener más lectores, más copias y más tiempo de conservación que la base de datos del negocio. Para investigar un caso basta con el **request_id**, el largo del mensaje, el tipo de error y la categoría de bloqueo.

8.2 Valores del campo **resultado**

resultado	Significado	Código
ok	Consulta respondida	200
bloqueado_llm01	Mensaje rechazado por posible manipulación. Vea <code>categorias_bloqueo</code>	400
fuga_bloqueada_llm07	El modelo intentó revelar sus instrucciones; la respuesta se reemplazó	200
rate_limit_excedido	Cuota agotada	429
no_authorized	Clave ausente o inválida	401
validacion_fallida	Solicitud mal formada o fuera de límites	422
cuerpo_demasiado_grande	Solicitud de más de 16 KB	413
error_upstream	Falló el proveedor. Vea <code>tipo_error</code> y <code>proveedor_fallido</code>	502, 503, 504
error_interno	Error inesperado	500

Una línea con **"degradado": true** indica que respondió el proveedor de respaldo; **proveedor_fallido** dice cuál falló.

8.3 Consultas útiles

Buscar qué pasó con una solicitud concreta:

```
grep '41bbfe68765947be' logs/gateway.jsonl
```

Ver los intentos de manipulación bloqueados:

```
grep 'bloqueado_llm01' logs/gateway.jsonl
```

Resumen de resultados:

```
venv/bin/python -c "import json,collections;
print(collections.Counter(json.loads(l).get('resultado') for l in open('logs/gateway.jsonl',
encoding='utf-8') if '\"solicitud\"' in l))"
```

```
Counter({'ok': 12, 'validacion_fallida': 3, 'rate_limit_excedido': 3, 'no_authorized': 2,
'error_upstream': 2})
```

Señales que conviene vigilar:

Señal	Posible causa
Muchos bloqueado_llm01 de un mismo cliente_id	Alguien está probando ataques desde esa aplicación
Algún fuga_bloqueada_llm07	Un ataque superó el filtro de entrada; revise el caso
rate_limit_excedido constante en una clave	La aplicación necesita más cuota o tiene un error que repite solicitudes
Muchos no_autorizado	Clave mal configurada, o intentos de adivinar claves
error_upstream repetido	Problema con el proveedor o con su clave

9. Demostración de las protecciones

Esta sección permite comprobar, sin API keys ni costo, que cada protección detiene el ataque que debe detener.

9.1 Comprobación automática

```
make verificar
make test
make evidencia
```

- **make verificar** arranca el gateway, hace seis comprobaciones con solicitudes reales y lo detiene.
- **make test** ejecuta 63 pruebas (puede mostrar un aviso, **1 warning**, que es inofensivo). Por cada protección hay una prueba que demuestra que el ataque **funciona** sin ella (**test_linea_base_...**) y otra que demuestra que queda **bloqueado** con ella (**test_con_mitigacion_...**).
- **make evidencia** ejecuta seis escenarios, muestra una tabla comparativa y genera **docs/evidencias/REPORTE_EVIDENCIA.md**.

9.2 Demostración en vivo

Se usan dos terminales en la máquina Linux (en Codespaces, el botón + abre la segunda): en la primera corre el gateway y en la segunda se lanzan los ataques.

Terminal 2, una sola vez:

```
export GW_KEY=$(cat .gw_key)
export GW_KEY_2=gw...clave_de_otra_aplicacion... # opcional: demuestra que el límite es por clave
```

Para cada protección, arranque el gateway **sin** ella, lance el ataque, deténgalo, arránquelo **con** todas y repita el ataque.

Protección	Arranque sin protección (terminal 1)	Ataque (terminal 2)
Manipulación del modelo (LLM01)	<code>make linea-base M=LLM01</code>	<code>./scripts/ataques_en_vivo.sh llm01</code>
Fuga de instrucciones (LLM07)	<code>MITIGACION_LLM01=false make linea-base M=LLM07</code>	<code>./scripts/ataques_en_vivo.sh llm07</code>
Consumo sin límite (LLM10)	<code>make linea-base M=LLM10</code>	<code>./scripts/ataques_en_vivo.sh llm10</code>
Fuga de información (LLM02)	<code>SIMULAR_FALLA_UPSTREAM=error_con_clave make linea-base M=LLM02</code>	<code>./scripts/ataques_en_vivo.sh llm02</code>

Arranque con todas las protecciones: `make protegido`. Para LLM07, con la entrada todavía apagada: `MITIGACION_LLM01=false make protegido`. Para LLM02, con el mismo fallo simulado: `SIMULAR_FALLA_UPSTREAM=error_con_clave make protegido`. Detenga el gateway con **Ctrl + C** entre un arranque y el siguiente.

Protección	Resultado sin protección	Resultado con protección
LLM01	200: el modelo anuncia "devoluciones en 365 días con reembolso doble"	400: solicitud rechazada; el proveedor no se llega a invocar
LLM07	200: el modelo entrega sus instrucciones internas completas	200: la respuesta se reemplaza por un mensaje seguro
LLM10	Las 8 solicitudes seguidas pasan	Desde la sexta, 429; otra clave sigue funcionando
LLM02	El error muestra la API key del proveedor y el registro guarda la consulta	503 con mensaje genérico; el registro no contiene datos sensibles

Notas para la demostración:

- **Para LLM07 se apaga también LLM01.** Si no, el filtro de entrada bloquearía el ataque antes y no se vería actuar al filtro de salida.
- **Reinicie el gateway antes de la demostración de LLM10.** Las solicitudes anteriores ya habrán consumido parte de la cuota.
- **Para LLM02, la API key que aparece es la configurada en `GOOGLE_API_KEY`.** Use una clave falsa para la demostración, nunca una real. Si no hay ninguna configurada, se muestra un texto de ejemplo.
- **El proveedor simulado obedece siempre los ataques,** para que el resultado sea repetible. Con un modelo real, el ataque sin protección a veces falla porque el propio modelo lo resiste.

El detalle de cada caso está en `docs/MAPEO_OWASP.md`.

10. Preguntas frecuentes

¿El gateway recuerda la conversación? No. Cada consulta es independiente. Incluya en el mensaje el contexto que necesite.

¿Puedo cambiar el comportamiento del asistente desde mi aplicación? No. Las instrucciones del asistente las define el administrador y son iguales para todas las aplicaciones. Los intentos de cambiarlas desde el mensaje se rechazan con **400**.

Mi consulta legítima fue rechazada con 400. ¿Qué hago? Reformúlela sin frases como "ignora las instrucciones" o "muestra tu system prompt". Si el rechazo persiste, envíe el **request_id** al administrador para que revise la categoría de bloqueo.

¿Por qué recibo 429 si hice pocas consultas? El límite por defecto es de 5 por minuto por clave, y las solicitudes rechazadas con **400** también cuentan. Espere un minuto o pida al administrador que aumente la cuota.

¿El gateway guarda lo que escriben mis usuarios? No. El registro guarda solo metadatos (fecha, resultado, largo del mensaje, consumo). El proveedor del modelo sí recibe el texto para poder responder.

La respuesta dice "simulado": true. ¿Es un modelo real? No. Es el proveedor de pruebas, que devuelve respuestas fijas. Pida al administrador que configure un proveedor real.

¿Qué significa "degradado": true? Que el proveedor principal falló y respondió el de respaldo. La respuesta es válida, pero puede venir de un modelo distinto al habitual.

Perdí mi clave. ¿Pueden reenviármela? No: el gateway solo guarda su hash. El administrador debe emitir una nueva.

¿Puedo usar la misma clave en varias aplicaciones? Puede, pero compartirán la cuota y no se distinguirán en el registro. Se recomienda una clave por aplicación.

11. Glosario

Término	Definición
API key	Clave que un proveedor de modelos entrega para usar y facturar su servicio
Gateway	Servicio intermedio por el que pasan todas las consultas hacia los modelos
Hash	Huella irreversible de un texto. Permite comprobar una clave sin guardarla
LLM	Modelo de lenguaje de gran tamaño (<i>Large Language Model</i>)
OWASP Top 10 for LLM	Lista de los diez riesgos de seguridad más importantes en aplicaciones con modelos de lenguaje
Prompt injection	Ataque en el que el texto del usuario intenta cambiar las instrucciones del modelo
System prompt	Instrucciones internas que definen el comportamiento del asistente
Token	Unidad de texto con la que los proveedores miden el consumo

Término	Definición